MAKERERE UNIVERSITY | COLLEGE OF COMPUTING & INFORMATION SCIENCES

STEP-BY-STEP ACTION PLAN & TECHNICAL ROADMAP

Engineering Implementation & Management Plan (Phases 0 to 5)

Kayeera Nathan (24/U/25913/EVE) - Group Leader, System Architect & Integrator

STEP-BY-STEP ACTION PLAN & TECHNICAL ROADMAP

Student: Kayeera Nathan (Reg. No: 24/U/25913/EVE)

Role: Group Leader, System Architect, and Integrator

Group: SE-24 | Makerere University (CoCIS) - BSE III & CS II

Course: Embedded Systems Coursework (Academic Year 2026/2027)

Project: Smart Control and Monitoring System for CoCIS

Communication Bus: Wireless ESP-NOW (2.4 GHz Peer-to-Peer Wi-Fi)

Submission Deadline: Monday, 2 November 2026 (Submit on MUELE)

Presentation / Viva: Monday, 2 November 2026 (In-person attendance mandatory)

1. Executive Summary & Your Dual Role

As Group Leader, System Architect, and Integrator, you have two equally vital responsibilities:

- 1. Technical Leadership & Architecture: You own the overarching system design, the wireless ESP-NOW (2.4 GHz Wi-Fi) communication layer, FreeRTOS concurrency and synchronization architecture, emergency evacuation logic, offline heartbeat monitoring, and the master integration of all 5 group members' code on ESP-IDF and Wokwi.
- 2. Project & Submission Governance: You are responsible for ensuring the group meets the 2 November deadline, freezing code on time, compiling the final comprehensive report, verifying all individual video links and Wokwi simulation links, and preparing the team to defend their work before the lecturer.

TIP / BEST PRACTICE

Why ESP-NOW Wi-Fi is our Primary Architectural Choice: In a real-world facility like CoCIS, laying hundreds of meters of physical UART wires between the main entrance and multiple lecture rooms across corridors is prone to electrical noise, signal degradation, and high installation costs. We replace physical serial cables with **ESP-NOW**, Espressif's connectionless 2.4 GHz Wi-Fi protocol. This operates without a router or internet access, delivers sub-5ms latency, includes hardware MAC-layer acknowledgments, and directly satisfies the coursework's heavy emphasis on **system innovation**.

2. Master Step-by-Step Execution Plan (Day-by-Day)

Phase 0: Kickoff, Setup & Interface Contract (Oct 23 - Oct 24) - *CURRENT STAGE*

☐ Step 0.1: Establish Team Governance & Confirm Roles

- Hold a kickoff call/chat with all 4 group members:
- Kasozi David Paul (24/U/25905/EVE): Entrance Terminal (RFID, doors, anti-passback, lockout).
- Zziwa Mark Alex (24/U/11978/EVE): Room Terminals (PIR motion, lighting relays, LDR light sensing, ESP-NOW transmitter).
- Sseku Jovan (24/U/11172/EVE): Storage & Data Management (SD card, CSV logs, timestamps, crash recovery).
- Namuzibwa Laurinda C (24/U/1472): CLI interface, Authentication, Security, QA & Test Plan.

- Set a fixed 15-minute daily standup meeting (e.g., 8:00 PM daily).

☐ Step 0.2: Initialize the Version Control Repository

- Create a private GitHub repository named SE-24-CoCIS-Smart-System.
- Set up directory structure:

```
components/common_protocol/ # Shared headers, ESP-NOW protocol, data types, FreeRTOS sy
src/
    ENTERANCE_MASTER/       # Entrance Master (ESP32 #1: RFID, SD, CLI, ESP-NOW Gateway
    ROOM_NODE_1/            # Lecture Room 1 (ESP32 #2 + ESP-NOW)
    ROOM_NODE_2/            # Lecture Room 2 (ESP32 #3 + ESP-NOW)
docs/                       # Diagrams, design notes, test logs
simulation/                 # Wokwi diagram.json, wokwi.toml, and virtual SD card
```

- Create dedicated feature branches for each member (feat/entrance-david, feat/rooms-mark, feat/storage-jovan, feat/cli-laurinda, feat/arch-nathan). Enforce a rule: *no direct pushes to main*.

☐ Step 0.3: Environment Setup & Wokwi Wireless Feasibility Check

- Confirm every member has ESP-IDF installed and can run a basic C application in Wokwi.
- Critical Technical Feasibility Checks for you to supervise:
- *Multi-ESP32 ESP-NOW Simulation:* Set up two ESP32 boards in Wokwi's diagram.json. Wokwi simulates the 2.4 GHz radio space automatically-no inter-board connecting wires are required! Run a simple ESP-NOW sender/receiver test to verify packet delivery in browser simulation.
- *RFID in Wokwi:* Verify David's RFID approach. If SPI MFRC522 has simulation constraints in Wokwi, implement the approved fallback: UART/Serial RFID tag emulation or keypad input, clearly documented.
- *SD Card in Wokwi:* Verify Jovan's SPI SD card simulation with FATFS/VFS.

☐ Step 0.4: Author and Freeze the Wireless Interface Contract ('design.md')

- Write down the exact pin allocations, ESP-NOW packet framing format, MAC address mapping, CSV data schemas, and FreeRTOS queue definitions.
- Freeze this document by Saturday midnight so your teammates can write independent code without integration headaches.

Phase 1: Nathan's Core Technical Engineering (Oct 25 - Oct 27)

During this phase, each teammate builds their unit modules. Here is your specific coding workload:

☐ Step 1.1: Develop the Wireless ESP-NOW Protocol Layer ('common/esp_now_transport.h' & '.c')

- Implement the connectionless Wi-Fi stack initialization:
- nvs_flash_init() (mandatory for ESP32 Wi-Fi RF calibration).
- esp_netif_init() and default event loop creation.
- Wi-Fi initialization in Station mode (WIFI_MODE_STA) and start RF without connecting to an SSID.
- esp_now_init() and registration of send and receive callbacks.
- Implement Peer Management:
- Register Master MAC address in Room nodes.
- Register Room 1 and Room 2 MAC addresses in Master node.
- Implement custom message frames within ESP-NOW payload (up to 250 bytes):
- MSG_ROOM_STATUS (Room -> Master: Occupied flag, Light flag, Lux level, energy timer).
- MSG_LIGHT_OVERRIDE (Master -> Room: Auto, Force OFF, Force ON).
- MSG_HEARTBEAT (Room -> Master: Periodic keep-alive ping every 2 seconds).
- MSG_EMERGENCY_ALARM (Master -> Broadcast: Instant evacuation alert).
- Implement Delivery Verification:
- Utilize ESP-NOW's built-in send_cb status (ESP_NOW_SEND_SUCCESS vs ESP_NOW_SEND_FAIL).
- Add software sequence IDs and automatic 3x retry on packet failure.

☐ Step 1.2: Build the Thread-Safe Concurrency & Synchronization Module ('common/sync_manager.h')

- The assignment explicitly grades Resource Synchronization and System Reliability (Section 2.7). You must build this properly using FreeRTOS primitives:
- Mutexes:
- xOccupancyMutex: Protects the in-memory array of persons currently inside the building.
- xScheduleMutex: Protects current operating hours and configuration state.
- xSDCardMutex: Protects shared SPI bus and file operations.
- xRoomStateMutex: Protects the cached telemetry states of Room 1 and Room 2.
- Queues:
- xAccessEventQueue: David's RFID task pushes entry/exit requests -> Master logic evaluates -> logs to SD.
- xSDLogQueue: Dedicated queue consumed by Jovan's SD logging task to eliminate disk write bottlenecks.
- xWirelessPacketQueue: ESP-NOW receive callback copies incoming packets into this queue, allowing the master processing task to handle wireless telemetry asynchronously without blocking Wi-Fi interrupts.

☐ Step 1.3: Code the Wireless Heartbeat & Offline Detection Engine

- Master runs a periodic FreeRTOS task (heartbeat_monitor_task, runs every 2-3 seconds).
- Tracks timestamp of last received wireless packet from Room 1 and Room 2.
- If no ESP-NOW heartbeat is received within 10 seconds, trigger WIRELESS_NODE_OFFLINE on the master console and flag the room as offline.

☐ Step 1.4: Conduct Peer Reviews (Tuesday Evening, Oct 27)

- Review David's Entrance code, Mark's Room code, Jovan's SD Card code, and Laurinda's CLI/Auth code.
- Verify that no teammate is writing directly to shared memory without using your mutexes, and all inter-task communication uses your FreeRTOS queues.

Phase 2: Master Node Integration & System Synchronization (Oct 28 - Oct 29)

☐ Step 2.1: Merge Teammate Modules into Node A (Master)

- Integrate David's RFID reader & door servo control into the master processing loop.
- Connect Laurinda's Serial CLI parser to your synchronized data tables.
- Connect Jovan's SD card manager so all door transitions write safely to access_log.csv.

☐ Step 2.2: Implement the Strict Building Access Schedule Logic

- Coursework Requirement (Section 2.6):
- Configurable time window (e.g., 07:00 to 20:00).
- Outside operating hours: Entry is denied.
- Crucial Rule: People inside the building CANNOT leave outside authorized hours (strict lockdown prevention against unauthorized after-hours asset removal).
- Exception: Authenticated Admin / Security override.

☐ Step 2.3: Integrate Wireless Room Telemetry with CLI Dashboard

- Connect incoming ESP-NOW packets from Room 1 & Room 2 to your local state cache.
- Master dashboard commands (rooms, lights, status) display live wireless telemetry received from the lecture room microcontrollers.

☐ Step 2.4: Execute Concurrency & Race-Condition Stress Tests

- Test simultaneous entry and exit scans happening at the exact same millisecond.
- Test a serial CLI query (occupancy or logs) while an SD card write is actively executing and an ESP-NOW wireless packet arrives.
- Ensure zero deadlocks, zero buffer corruptions, and zero missed log entries.

Phase 3: Innovation Features, Hardening & Code Freeze (Oct 29 - Oct 30)

The coursework places particular emphasis on innovative features. You lead and verify these:

☐ Step 3.1: Finalize Your Personal Innovation - Wireless Emergency Evacuation Mode

- CLI command emergency activate or a dedicated physical emergency pushbutton.
- Instantly unlocks both door servos (servos swing to 90 open position).
- Broadcasts wireless MSG_EMERGENCY_ALARM via ESP-NOW to all lecture rooms (forces all room lights ON for emergency evacuation visibility).
- Flushes occupancy records to SD card with an EMERGENCY_EVACUATION event log.
- Command emergency reset broadcasts normal status and restores regular schedule.

☐ Step 3.2: Coordinate Teammate Innovations

- Wireless ESP-NOW Smart Mesh (Nathan): Cable-free building automation network with sub-5ms latency and automatic peer health monitoring.
- Anti-passback (David & Jovan): Tag cannot enter if already inside; cannot exit if already outside.
- Intrusion Lockout (David & Laurinda): 3 consecutive invalid RFID scans trigger buzzer alarm and lock terminal for 60 seconds.
- Daylight-Aware Smart Lighting (Mark): If room has high ambient light (LDR reading high), lights remain off even if PIR senses occupancy.
- Crash Recovery (Jovan): On master reboot/power-cut, master reads occupancy.csv and config.txt to restore active occupants and schedule without data loss.
- Energy Savings Analytics (Mark & Jovan): Calculates and prints kilowatt-hours/minutes saved when lights were shut off due to no occupancy.

☐ Step 3.3: Complete Full System Verification Against Test Matrix

- Execute all test cases with Laurinda (Valid entry, Invalid exit, Anti-passback trigger, Outside schedule entry/exit attempt, Wireless packet packet drop test, Power reboot recovery, SD card removal handling).

☐ Step 3.4: HARD CODE FREEZE - Friday, 30 October 2026 at 8:00 PM

- Merge all stable feature branches into main.
- Lock the repository. *No code edits after this point except urgent bug fixes.*

Phase 4: Deliverables Assembly, Video Recordings & Report (Oct 30 - Nov 1)

According to Section 4 of the Coursework PDF, each group must submit 5 mandatory deliverables. Here is your checklist:

☐ Deliverable 1: Comprehensive System Operation Report

- Coordinate report compilation with Laurinda:
- *Nathan’s written sections:* Section 1: System Overview & Architecture; Section 2: Wireless ESP-NOW Protocol & Peer Network; Section 3: FreeRTOS Concurrency & Synchronization; Section 6: Wireless Emergency Evacuation & Heartbeat Innovations; Section 7: System Integration & Master Wokwi Setup.
- Include block diagrams, state machine flowcharts, circuit schematics, and concurrency mutex diagrams.
- Include the table of group members and clear individual contributions.

☐ Deliverable 2: Public Wokwi Simulation Link

- Build the clean, consolidated Wokwi project with all 3 ESP32 nodes running wirelessly.
- Ensure the simulation link is set to Public and test it in an Incognito/Private browser window.

☐ Deliverable 3: Individual Video Demonstrations (Screen Recordings)

- Each member MUST record their own 3 to 5-minute video.
- Nathan’s Personal Video Script (3-5 min):
- 1. *Introduction (30s):* Name, Reg No, Role (Leader & Architect), brief overview of the 3-node distributed wireless system.
- 2. *Architecture & Synchronization (1.5 min):* Show the block diagram, explain the FreeRTOS mutexes, message queues, and why synchronization prevents race conditions.
- 3. *Wireless ESP-NOW Protocol & Heartbeat (1 min):* Show the C code in esp_now_transport.c, demonstrate packet send/receive callbacks, hardware MAC ACKs, and wireless node offline detection.
- 4. *Live Wokwi Demonstration (1.5 min):* Trigger the Emergency Evacuation Mode live in Wokwi, show doors unlocking, wireless broadcast packet triggering lecture room lights ON, and emergency log entry written to SD card.
- Upload your video to Google Drive / YouTube (unlisted) and ensure link access is set to *"Anyone with the link can view"*.

☐ Deliverable 4: Individual Contributions Section

- Ensure your written report explicitly details every member's exact contribution to avoid mark deductions.

☐ Deliverable 5: Group Details Table

- Ensure all 5 names, registration numbers, and roles are properly formatted on the title page.

☐ Step 4.5: Conduct Team Mock Defense (Sunday, 1 Nov)

- Run a mock Q&A session. Quiz David on anti-passback, Mark on PIR debounce and ESP-NOW transmissions, Jovan on FATFS queues, and Laurinda on SHA/hash authentication.
- Ensure everyone understands how the wireless system fits together.

Phase 5: Final Submission & Lecturer Defense (Nov 1 - Nov 2)

☐ Step 5.1: Final MUELE Submission (Sunday Night / Early Monday Morning)

- Export the report as a clean PDF (SE24_CoCIS_Smart_Control_System_Report.pdf).
- Verify all 5 video links and the Wokwi link are live and clickable inside the PDF.
- Upload the final report to MUELE well before the deadline on 2nd November 2026.

☐ Step 5.2: In-Person Presentation & Defense (Monday, 2 November 2026)

- Ensure 100% group attendance (as stated in coursework instructions, absent members get zero marks).
- Open the live Wokwi simulation on a laptop, ready to demonstrate.
- Confidently explain your architecture, wireless ESP-NOW peer-to-peer design, synchronization mechanisms, and leadership coordination.

3. Kayeera Nathan's Technical Specification & Code Blueprint

3.1 Wireless ESP-NOW Protocol Definition ('common/protocol.h')

```
#ifndef PROTOCOL_H
#define PROTOCOL_H

#include <stdint.h>
#include <stdbool.h>

#define ESP_NOW_PAYLOAD_MAGIC 0xAA55

typedef enum {
    NODE_MASTER  = 0x01,
    NODE_ROOM_1  = 0x02,
    NODE_ROOM_2  = 0x03
} node_id_t;

typedef enum {
    MSG_TYPE_HEARTBEAT       = 0x01,
    MSG_TYPE_ROOM_STATUS     = 0x02,
    MSG_TYPE_LIGHT_OVERRIDE  = 0x03,
    MSG_TYPE_EMERGENCY_ALERT = 0x04
} msg_type_t;

// ESP-NOW Wireless Packet Frame (Fits easily within ESP-NOW 250-byte max)
typedef struct __attribute__((packed)) {
    uint16_t magic;           // 0xAA55 Protocol Identifier
    uint8_t  msg_id;          // Sequence ID for tracking
    uint8_t  sender_id;       // Node ID of sender
    uint8_t  target_id;       // Node ID of receiver (or 0xFF for broadcast)
    uint8_t  msg_type;        // Type of message
    uint8_t  payload_len;     // Length of payload
    uint8_t  payload[32];     // Application payload
    uint8_t  checksum;        // XOR checksum for payload integrity
```

```
} esp_now_packet_t;

// Room status payload structure
typedef struct __attribute__((packed)) {
    uint8_t  occupied;        // 1 = Occupied, 0 = Empty
    uint8_t  lights_on;       // 1 = ON, 0 = OFF
    uint16_t light_level_lux; // Ambient brightness from LDR
    uint32_t seconds_active;  // Energy saving counter
} room_status_payload_t;

#endif
```

3.2 FreeRTOS Concurrency Architecture (‘src/ENTERANCE_MASTER/main.c‘)

```
/*
 * CONCURRENCY CONTROL & WIRELESS PACKET DISPATCH
 * Designed by: Kayeera Nathan (System Architect, SE-24)
 */

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "freertos/queue.h"
#include "esp_now.h"
#include "protocol.h"

// Mutexes for Thread Safety
SemaphoreHandle_t xOccupancyMutex;
SemaphoreHandle_t xScheduleMutex;
SemaphoreHandle_t xSDCardMutex;
SemaphoreHandle_t xRoomStateMutex;

// Event and Log Queues
QueueHandle_t xAccessEventQueue;
QueueHandle_t xSDLogQueue;
QueueHandle_t xWirelessPacketQueue;

typedef struct {
    char tag_id[16];
    char user_name[32];
    uint8_t direction;        // 0 = ENTRY, 1 = EXIT
    uint32_t timestamp;
} access_event_t;

void init_concurrency_objects(void) {
    xOccupancyMutex      = xSemaphoreCreateMutex();
    xScheduleMutex       = xSemaphoreCreateMutex();
    xSDCardMutex         = xSemaphoreCreateMutex();
    xRoomStateMutex      = xSemaphoreCreateMutex();

    xAccessEventQueue    = xQueueCreate(10, sizeof(access_event_t));
    xSDLogQueue          = xQueueCreate(20, sizeof(access_event_t));
    xWirelessPacketQueue = xQueueCreate(15, sizeof(esp_now_packet_t));
}

// Wireless ESP-NOW RX Callback - pushes packet to queue without blocking Wi-Fi
void on_esp_now_data_recv(const esp_now_recv_info_t *info, const uint8_t *data, int len) {
    if (len == sizeof(esp_now_packet_t)) {
        esp_now_packet_t pkt;
        memcpy(&pkt, data, sizeof(esp_now_packet_t));
        BaseType_t xHigherPriorityTaskWoken = pdFALSE;
        xQueueSendFromISR(xWirelessPacketQueue, &pkt, &xHigherPriorityTaskWoken);
        if (xHigherPriorityTaskWoken) {
```

```
            portYIELD_FROM_ISR();
        }
    }
}
```

4. Immediate Next Steps (To Do Today, Oct 24)

- 1. Share the role confirmation with the group chat.
- 2. Initialize the GitHub repository SE-24-CoCIS-Smart-System.
- 3. Save the frozen INTERFACE_CONTRACT_AND_DESIGN.md specification and announce the upgrade to ESP-NOW Wi-Fi.
- 4. Test the multi-ESP32 ESP-NOW simulation in Wokwi (no interconnecting wires needed!).